Расчетно-аналитическое задание

На тему: «Метрический классификатор»

Цель

Основной целью работы является приобретение знаний о метрических классификаторов, а также навыков построения данного класса моделей в рамках своих исследований.

Задачи:

- освоить теоретические знания в части метрик расстояний и метрических классификаторов;
- Получить практические навыки создания моделей метрических классификаторов на основе методов k-ближайших соседей и парзеновских окон.

Метрики расстояний

В науке о данных часто необходимо определить меру схожести между двумя объектами. Схожесть или похожесть необходимо оценивать во многих задачах, в том числе при выявлении предпочтений пользователей, определении плагиата контента (текста, картин и т.д.), решении задач кредитного скоринга (принимаем гипотезу о том, что большинство похожих по основным признакам клиентов банка с одинаковой вероятностью вернут или не вернут кредит), и т.д. Важным условием в контексте описываемой задачи является компактность исследуемых объектов. То есть похожие объекты находятся достаточно близко друг к другу, а не похожие как можно дальше. В качестве примера можно привести ирисы Фишера на рисунке 1.

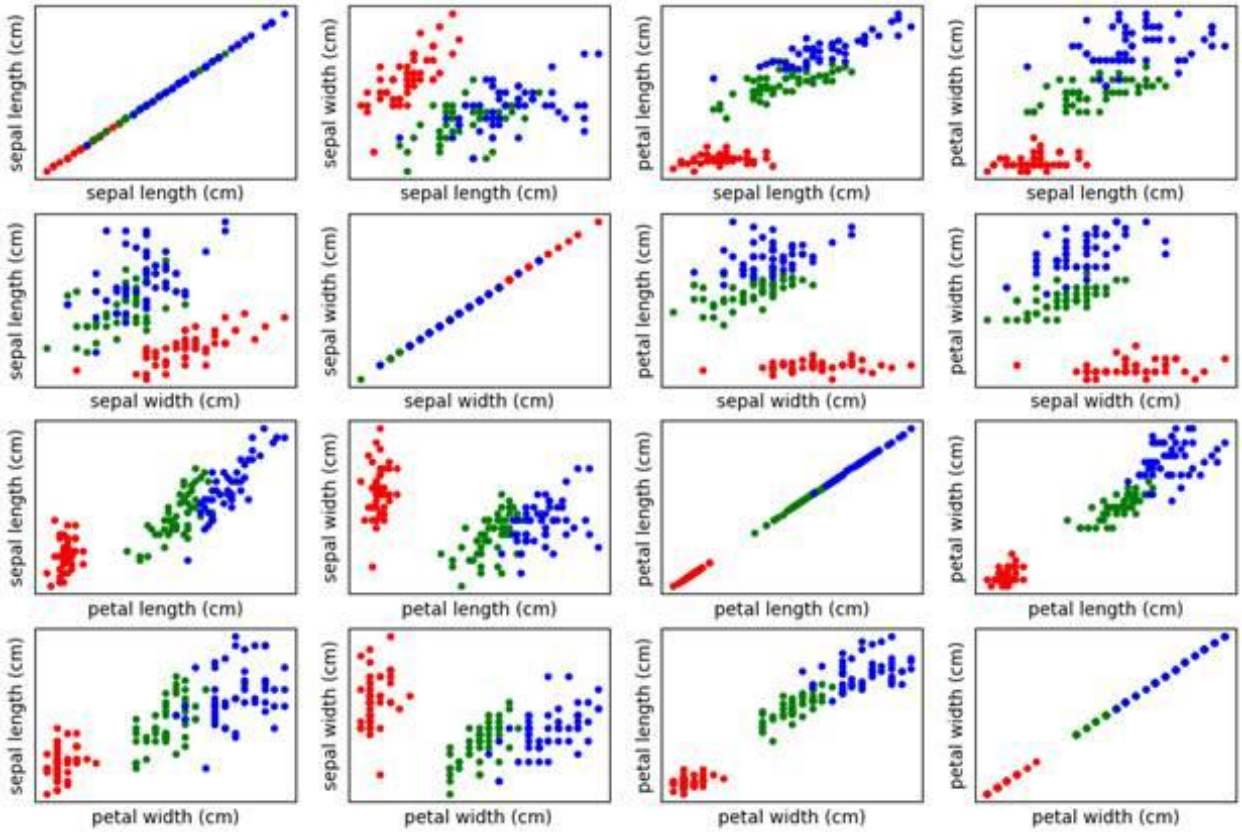


Рисунок 1. Зависимость признаков ирисов Фишера

Как видно на рисунке 1, ирисы одного типа (каждый тип ириса обозначен цветом точек, красный зеленый или синий) расположены близко и образуют компактные скопления точек.

В связи с необходимостью решения описанной задачи необходимо задать некоторую функцию в признаковом пространстве x_i, в котором заданы исследуемый объект x, которая позволит оценить степень похожести этих объектов. В связи с этим в математике введено понятие метрики расстояния $\rho(x, x_i)$.

Метрика расстояния — это функция расстояния на множестве объектов, которая каждой паре объектов ставит в соответствие неотрицательное число и удовлетворяет аксиомам тождества, симметрии и неравенства треугольника. [http://www.machinelearning.ru/wiki/index.php?title=%D0%9C%D0%B5%D1%82%D1%80%D0%B8%D0%BA%D0%B0].

Однако, не любая математическая функция может выступать в качестве метрики расстояния. Для того, чтобы функция считалась метрикой расстояния, она должна удовлетворять следующим свойствам:

1. неотрицательность, т.е. $\rho(x, x_i) \geq 0$;
2. равенство метрики нулю **тогда и только тогда**, когда объекты совпадают: $\rho(x, x_i) = 0 \Leftrightarrow x = x_i$;
3. симметричность: $\rho(x, x_i) = \rho(x_i, x)$;
4. Неравенство треугольника (расстояние напрямую от x до z не может быть больше, чем сумма расстояний от x до y и от yдо z.):

$$\rho(x, z) \leq \rho(x, y) + \rho(y, z).$$

В машинном обучении применяют 3 наиболее распространенные метрики расстояния: Евклидова, Манхеттена, Чебышева.

Евклидово расстояние

Евклидово расстояние — это самое обычное «прямолинейное» расстояние между двумя точками в пространстве. Именно это расстояние измеряется линейкой в реальной жизни. Оно основано на теореме Пифагора и используется в разнообразных областях науки, в том числе в машинном обучении. Графическое представление Евклидова расстояния представлена на рисунке 2.

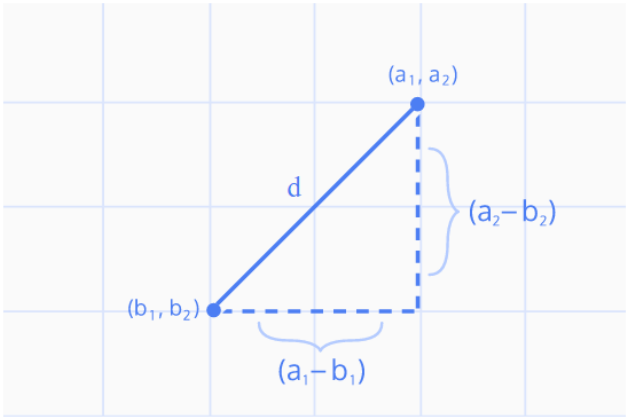


Рисунок 2. Графическое представление Евклидова расстояния

Пусть в пространстве признаков x^j задано n объектов:

$x_i = [x_i^1, x_i^2, \ldots, x_i^j]$, $i = 1,2,\ldots,l$, *(i – номер объекта, j – номер признака)*

В заданном пространстве объектов также задан объект $x = [x^1, x^2, \ldots, x^l]$

Чтобы найти Евклидово расстояние между i-м объектом из x^j и заданным объектом x необходимо применить следующее выражение:

$$\rho(x, x_i) = \sqrt{\sum_{j=1}^{n}(x^j - x_i^j)^2} = \left(\sum_{j=1}^{n}(x^j - x_i^j)^2\right)^{1/2}$$

Рассмотрим простейший пример. Допустим, у нас есть две точки на плоскости:

Точка A: $(1, 2)$
Точка B: $(4, 6)$
Шаг 1: находим разницу по каждой оси.
По оси X: $4 - 1 = 3$
По оси Y: $6 - 2 = 4$
Шаг 2: возводим разницы в квадрат.

$$3^2 = 9$$
$$4^2 = 16$$

Шаг 3: складываем квадраты и извлекаем квадратный корень.

$$d = \sqrt{9 + 16} = \sqrt{25} = 5$$

Ответ: Евклидово расстояние между точками A и B равно **5**.

Теперь рассмотрим, как можно рассчитать Евклидово расстояние средствами языка программирования python. В Python есть несколько способов вычислить евклидово расстояние.

Способ 1: Встроенная функция math.dist

Модуль math в Python — это встроенная библиотека, которая предоставляет доступ к математическим функциям и константам из стандарта языка C. Данный модуль поставляется по умолчанию вместе с языком программирования python, однако его необходимо импортировать для использования в рамках разрабатываемого вами модуля. В библиотеке math существует встроенная функция math.dist(). Это стандартная функция из модуля math, появившаяся в Python 3.8 специально для вычисления расстояний между точками.

Данная функция принимает два итерируемых объекта (списки, кортежи) одинаковой длины и возвращает Евклидово расстояние.

Рассмотрим пример на python. Для начала импортируем модуль math, который не требует дополнительной установки, и зададим исходные точки из предыдущего примера как кортежи на python.

```
import math

point_a = (1, 2)
point_b = (4, 6)
```

Далее рассчитаем и выведем в консоль полученное Евклидово расстояние между двумя точками:

```
distance = math.dist(point_a, point_b)
print(f"Расстояние: {distance}")  # Вывод: 5.0
```

А теперь давайте попробуем рассчитать расстояние между двумя точками, заданными в пространстве бОльшей размерностью.

```
point_3d_a = [1, 2, 3]
point_3d_b = [4, 5, 6]
distance_3d = math.dist(point_3d_a, point_3d_b)
print(f"Расстояние в 3D: {distance_3d}")
```

Как видите, используя данную функцию можно рассчитать Евклидово расстояние между двумя точками любой размерностью. Однако, точки должны быть заданы в одном и том же пространстве (размерность должна совпадать). При несовпадении размерности будет выведена ошибка, так как при несовпадении размерности сравниваемых точек невозможно рассчитать Евклидово расстояние.

Достоинства использования функции math.dist():

- не требует установки внешних библиотек;
- работает с любой размерностью (2D, 3D, n-D);
- принимает списки, кортежи и другие итерируемые объекты;
- читаемый код.

Недостатки использования функции math.dist():

- работает только с двумя точками (не умеет считать попарные расстояния между множествами точек);
- медленный при работе с большими массивами данных.

Способ 2. numpy.linalg.norm() – норма вектора

numpy.linalg.norm() – Функция из библиотеки NumPy для вычисления различных норм векторов. Евклидово расстояние — это L2-норма разности двух векторов.

При использовании данной функции вычисляется разность двух массивов (векторов), а затем находите норму полученного вектора. Рассмотрим пример на python. Библиотека NumPy не является встроенным модулем языка python и требует дополнительной установки. Прежде, чем писать свой модуль, проверьте, установлена ли данная библиотека:

```
pip install numpy
```

Когда модуль установлен, его можно импортировать внутри вашего кода. Импортируем NumPy и вводим все те же точки из примера выше:

```
mport numpy as np
```

```
point_a = np.array([1, 2])
point_b = np.array([4, 6])
```

Далее с использованием функции numpy.linalg.norm() рассчитаем Евклидово расстояние между заданными точками:

```
# Евклидово расстояние = L2-норма разности векторов
distance = np.linalg.norm(point_b - point_a)
print(f"Расстояние: {distance}")  # Вывод: 5.0

# Для 3D
point_3d_a = np.array([1, 2, 3])
point_3d_b = np.array([4, 5, 6])
distance_3d = np.linalg.norm(point_3d_b - point_3d_a)
print(f"Расстояние 3D: {distance_3d}")  # Вывод: 5.196...
```

Обратите внимание, что при использовании рассматриваемой функции NumPy необходимо задавать массивы np.array(). Данная функция является вычислительно оптимизированной, поэтому в машинном обучении (включая k-NN, метод Парзена и потенциальные функции) она используется в 99% случаев для быстрого вычисления Евклидова расстояния.

Основной синтаксис описываемой функции выглядит так:

```
np.linalg.norm(x, ord=None, axis=None, keepdims=False)
```

- x: входной массив (вектор или матрица).
- ord: порядок нормы. По умолчанию None (для векторов это L2-норма, т.е. евклидово расстояние).
- axis: ось, вдоль которой вычисляется норма. **Критически важно для матриц!**

Более подробно об этой функции можно прочитать в документации библиотеки NumPy.

Достоинства использования функции numpy.linalg.norm():

- очень быстро работает с большими массивами данных;
- поддерживает векторизацию (можно считать расстояния между множествами точек);
- стандарт для NumPy и Data Science;
- может вычислять другие нормы (L1, L∞ и т.д.).

Недостатки использования функции numpy.linalg.norm():

- требует установки NumPy;
- нужно явно вычислять разность векторов перед вызовом функции.

Способ 3. scipy.spatial.distance.euclidean()

SciPy — это бесплатная библиотека с открытым исходным кодом для языка Python, предназначенная для научных и инженерных расчетов.

Она построена на базе библиотеки NumPy и расширяет её возможности, предоставляя мощные алгоритмы для высшей математики, анализа данных и моделирования.

scipy.spatial.distance.euclidean() – специализированная функция из библиотеки SciPy для вычисления расстояний между точками. Данная функция принимает два массива (или списка) и возвращает Евклидово расстояние.

SciPy так же, как и NumPy, не является встроенным модулем python. Прежде, чем использовать его функционал необходимо установить данную библиотеку:

```
pip install scipy
```

Когда модуль установлен, его можно импортировать внутри вашего кода. Импортируем SciPy и вводим все те же точки из примера выше:

```
from scipy.spatial import distance

point_a = [1, 2]
point_b = [4, 6]
```

Далее с использованием функции scipy.spatial.distance.euclidean() рассчитаем Евклидово расстояние между заданными точками:

```
distance_val = distance.euclidean(point_a, point_b)
print(f"Расстояние: {distance_val}")  # Вывод: 5.0

# Работает с любой размерностью
point_3d_a = [1, 2, 3]
point_3d_b = [4, 5, 6]
distance_3d = distance.euclidean(point_3d_a, point_3d_b)
print(f"Расстояние 3D: {distance_3d}")  # Вывод: 5.196...
```

Достоинства использования функции scipy.spatial.distance.euclidean():

- семантически понятное имя функции;
- работает с любой размерностью;
- часть большой библиотеки SciPy с множеством других метрик расстояния.

Недостатки использования функции scipy.spatial.distance.euclidean():

- требует установки SciPy (тяжелее, чем просто NumPy);
- медленнее, чем np.linalg.norm для простых случаев;
- избыточна, если нужна только эта одна функция.

Способ 4. sklearn.metrics.pairwise_distances() – попарные расстояния

Scikit-learn (sklearn) – это популярная библиотека для языка Python, предназначенная для классического машинного обучения и анализа данных. Она построена на базе библиотек NumPy, SciPy и Matplotlib.

sklearn.metrics.pairwise_distances() – функция из scikit-learn для вычисления матрицы попарных расстояний между множествами точек. Данная функция принимает матрицу (набор точек) и возвращает матрицу расстояний между всеми парами точек.

Scikit-learn (sklearn) так же, как и NumPy, не является встроенным модулем python. Прежде, чем использовать его функционал необходимо установить данную библиотеку:

```
pip install scikit-learn
```

Так же стоит обратить внимание, что функции библиотеки в качестве входных данных принимают только массивы NumPy и Pandas. В рамках данного примера на вход зададим массив NumPy, поэтому на первом этапе импортируем необходимые модули и снова зададим исходные данные:

```
import numpy as np
from sklearn.metrics import pairwise_distances

# Матрица точек: каждая строка — одна точка
points = np.array([
    [1, 2],    # Точка A
    [4, 6],    # Точка B
    [0, 0]     # Точка C
])
```

Как видно из кода, в данном случае все точки задаются одним массивом, а не разными, как при использовании предыдущих способов. Это очень удобно при оценке расстояния в рамках больших наборов данных (датасетов). Далее вычисляем матрицу расстояний:

```
dist_matrix = pairwise_distances(points, metric='euclidean')


print("Матрица расстояний:")
print(dist_matrix)
# Вывод:
# [[0.         5.         2.23606798]
#  [5.         0.         7.21110255]
#  [2.23606798 7.21110255 0.        ]]

# Расстояние между точкой A (индекс 0) и точкой B (индекс 1)
print(f"A <-> B: {dist_matrix[0, 1]}")  # Вывод: 5.0
```

Функция pairwise_distances() принимает большое количество параметров. Здесь рассмотри только несколько из них:

x – входной набор данных (массив);

metric – название метрики, по которой следует осуществить расчет расстояния между точками. Функция поддерживает множество основных метрик, представленных в таблице:

Метрика	**Значение параметра metric**	**Когда использовать**
Евклидово	'euclidean' (по умолчанию)	Обычные числовые данные
Манхеттенское	'manhattan' или 'cityblock'	Устойчивость к выбросам, grid-данные

Косинусное	'cosine'	Тексты, embeddings, направления
Чебышева	'chebyshev'	Максимальная разница по одной оси
Минковского	'minkowski'	Обобщение (нужен параметр p)
Хэмминга	'hamming'	Бинарные/категориальные признаки
Jaccard	'jaccard'	Множества, бинарные векторы

Существует множество других способов расчета Евклидова расстояния с использованием языка программирования python, однако, представленных методов достаточно для решения задач машинного обучения.

Задание

1. Ниже представлены данные, описывающие три фильма. На основании этих данных рассчитать попарно Евклидово расстояние между фильмами и выявить наиболее похожие из них. Для расчета использовать 3 ранее рассмотренных методов.

Фильм	Длительность (мин)	Год выпуска	Бюджет ($)
Avatar (2009)	162	2009	237000000
Titanic (1997)	194	1997	200000000
The Dark Knight (2008)	152	2008	185000000

2. Создайте функции python для всех трех способов расчета. На вход функции принимают точки для сравнения, возвращают расстояния между всеми фильмами. Добавьте обязательную проверку на совпадении размерности пространств, в которых заданы входные данные.

Манхэттенское расстояние

Манхэттенское расстояние (также известное как расстояние городских кварталов, L1-норма, taxicab distance) — это расстояние между двумя точками, измеряемое по осям координат. Представьте, что вы едете на такси по Манхэттену — вы можете двигаться только вдоль улиц (по горизонтали и вертикали), но не по диагонали. Графическое представление Манхэттенского расстояния представлено на рисунке 3.

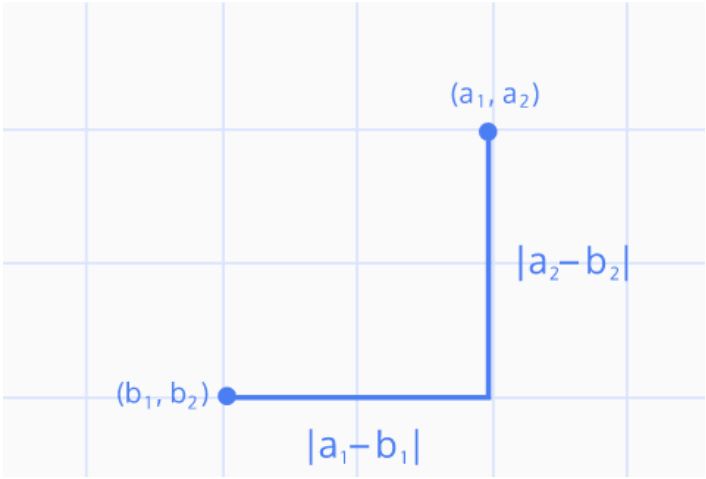


Рисунок 3. Графическое представление Манхэттенского расстояния

Используя обозначения из начала параграфа, запишем формулу Манхэттенского расстояния:

$$\rho(x,\ x_i)\ = \sum_{j=1}^{n} |x^j - x_i^j|$$

Рассмотрим пример расчета с использованием все тех же входных данных из раздела про Евклидово расстояние.

Точка A: $(1, 2)$
Точка B: $(4, 6)$
Шаг 1: находим разницу по каждой оси.
По оси X: $4 - 1 = 3$
По оси Y: $6 - 2 = 4$
Шаг 2: берем модули разниц.

$$| 3 |= 3$$
$$| 4 |= 4$$

Шаг 3: складываем модули.

$$d = 3 + 4 = 7$$

Ответ: Манхеттенское расстояние между точками A и B равно **7**.

Для сравнения: Евклидово расстояние между этими же точками было бы $\sqrt{3^2 + 4^2} = 5$. Манхеттенское расстояние всегда больше или равно Евклидовому.

Теперь рассмотрим, как можно рассчитать Манхэттенское расстояние средствами языка программирования python. В Python есть несколько способов вычислить манхэттенское расстояние. Данные способы аналогичны Евклидову расстоянию, поэтому рассмотрим их вкратце.

Способ 1: scipy.spatial.distance.cityblock()

Данная функция принимает два массива (или списка) и возвращает манхеттенское расстояние. Название "cityblock" отражает суть — расстояние по городским кварталам.

Пример на python:

```
from scipy.spatial import distance

point_a = [1, 2]
point_b = [4, 6]
```

```
distance_val = distance.cityblock(point_a, point_b)
print(f"Расстояние: {distance_val}")  # Вывод: 7

# Работает с любой размерностью
point_3d_a = [1, 2, 3]
point_3d_b = [4, 5, 6]
distance_3d = distance.cityblock(point_3d_a, point_3d_b)
print(f"Расстояние 3D: {distance_3d}")  # Вывод: 9
```

Способ 2: Использование NumPy

Существует несколько вариантов расчета расстояния Манхэттена с использованием библиотеки NumPy. Наиболее часть применяется следующий вариант кода для расчета Манхэттенского расстояния:

```
import numpy as np

point_a = np.array([1, 2])
point_b = np.array([4, 6])

# Манхеттенское расстояние = сумма модулей разностей
distance = np.sum(np.abs(point_b - point_a))
print(f"Расстояние: {distance}")  # Вывод: 7

# Для 3D
point_3d_a = np.array([1, 2, 3])
point_3d_b = np.array([4, 5, 6])
distance_3d = np.sum(np.abs(point_3d_b - point_3d_a))
print(f"Расстояние 3D: {distance_3d}")  # Вывод: 9
```

Вы вычисляете разность двух массивов (векторов), берете абсолютные значения (модули) и суммирует их.

Кроме того, можно рассчитать Манхэттенское расстояние аналогично Евклидову с использованием векторных норм, изменив параметр ord в значение 1 (норма L1).

```
import numpy as np

point_a = np.array([1, 2])
point_b = np.array([4, 6])

# Вычисляем L1-норму вектора разности
distance = np.linalg.norm(point_a - point_b, ord=1)

print(f"Манхеттенское расстояние: {distance}")  # Вывод: 7.0
```

Для очень простых одномерных случаев np.sum(np.abs()) может быть на доли микросекунды быстрее, так как np.linalg.norm имеет небольшой оверхед на проверку параметров, но на практике разница незаметна.

Способ 3: sklearn.metrics.pairwise_distances() — Попарные расстояния

Данная функция принимает матрицу (набор точек) и возвращает матрицу манхеттенских расстояний между всеми парами точек.

```python
import numpy as np
from sklearn.metrics import pairwise_distances

# Матрица точек: каждая строка — одна точка
points = np.array([
    [1, 2],    # Точка A
    [4, 6],    # Точка B
    [0, 0]     # Точка C
])

# Вычисляем матрицу попарных манхеттенских расстояний
dist_matrix = pairwise_distances(points, metric='manhattan')

print("Матрица манхеттенских расстояний:")
print(dist_matrix)
# Вывод:
# [[0. 7. 3.]
#  [7. 0. 10.]
#  [3. 10. 0.]]

# Расстояние между точкой A (индекс 0) и точкой B (индекс 1)
print(f"A <-> B: {dist_matrix[0, 1]}")  # Вывод: 7
```

Данный способ является аналогичным расчету Евклидово расстояния и более подробных пояснений не требует.

Задание

1. Аналогично предыдущему заданию разработать скрипт, включающий функции для расчета манхэттенского расстояния и проверить на заданных в рамках него данных (3 фильма).

Обобщение метрик

Обобщением рассмотренных метрик расстояний можно считать метрику Минковского:

$$\rho(x,\ x_i)\ = \left(\sum_{j=1}^{n} \omega_j |x^j - x_i^j|^p\right)^{1/p}$$

Чтобы эта функция удовлетворяла всем аксиомам метрики (в частности, неравенству треугольника), параметр pдолжен быть больше или равен 1 ($p \geq 1$)

При $p < 1$неравенство треугольника нарушается, и функция перестаёт быть строгой метрикой.

При $p = 1$ метрика превращается в расстояние Манхэттена, где сумма абсолютных разностей координат.

При $p = 2$ это классическое евклидово расстояние (прямая линия между двумя точками), наиболее привычное нам из школьной геометрии.

При $p \to \infty$ метрика стремится к расстоянию Чебышева, которое определяется как максимальная абсолютная разность по любой из координат: $\max_i | x_i - y_i |$

Задание

1. Самостоятельно напишите функцию, реализующую расчет расстояния Чебышева. Проверить на данных о фильмах.

Веса ω_j или стандартизация

В обобщенной метрике Минковского в формуле присутствует ω_j. Это так называемые веса, которые позволяют нормировать (масштабировать) входные данные.

Рассмотрим пример с фильмами из предыдущих разделов. Если рассчитать расстояние по "сырым" числам, признак с самыми большими абсолютными значениями (бюджет) или самыми большими разбросами обнулит вклад остальных признаков. Например, разница в 1 год будет математически "весить" больше, чем разница в 10 минут хронометража, что лишает анализ смысла.

В машинном обучении перед расчетом метрик расстояния данные всегда **нормализуют** (приводят к единому масштабу, например, от 0 до 1). Для этого используют два основных метода.

Метод 1. max-min стандартизация

Данный способ приводит данные к интервалу от 0 до 1 включительно (минимум становится равным 0, максимум равен 1). Для этого используется следующее выражение:

$$z_i^j = \frac{x_i^j - MIN(x^j)}{MAX(x^j) - MIN(x^j)}$$

x_i^j – исходное значение;

$MIN(x^j)$ – минимальное значение признака в наборе данных;

$MAX(x^j)$ – максимальное значение признака в наборе данных.

Рассмотрим реализацию на python.

Способ 1: Ручная реализация

```
import numpy as np

# Исходные данные: бюджеты фильмов (млн $)
budgets = np.array([237000000, 200000000, 185000000])

# Находим минимум и максимум
min_val = np.min(budgets)
max_val = np.max(budgets)
```

```
# Применяем формулу Min-Max
budgets_normalized = (budgets - min_val) / (max_val - min_val)

print("Исходные бюджеты:", budgets)
print("Нормализованные:", budgets_normalized)
# Вывод: Исходные бюджеты: [237000000 200000000 185000000]
Нормализованные: [1.         0.28846154 0.        ]
```

Способ 2: Использование sklearn.preprocessing.MinMaxScaler
Это стандартный подход в машинном обучении.

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Данные: [длительность, год, бюджет]
data = np.array([
    [162, 2009, 237],  # Avatar
    [194, 1997, 200],  # Titanic
    [152, 2008, 185]   # The Dark Knight
])

# Создаем scaler
scaler = MinMaxScaler()

# Обучаем и преобразуем (fit_transform)
data_normalized = scaler.fit_transform(data)

print("Нормализованные данные:")
print(data_normalized)
# Вывод:
# [[0.24 1.   1.  ]
#  [1.   0.   0.27777778]
#  [0.   0.91666667 0.  ]]

# Можно задать другой диапазон, например [-1, 1]
scaler_custom = MinMaxScaler(feature_range=(-1, 1))
data_custom = scaler_custom.fit_transform(data)
print("\nВ диапазоне [-1, 1]:")
print(data_custom)
```

Второй способ является классическим и повсеместно используется в машинном обучении.

Метод 2. Z-score нормализация (Стандартизация)

Z-score нормализация преобразует данные так, чтобы они имели среднее значение 0 и стандартное отклонение 1.

$$z_i^j = \frac{x_i^j - \mu^j}{\sigma^j},$$

где z_i^j - стандартизированное значение j-го признака для i-го объекта, μ^j — среднее значение j-го признака, а σ^j — его стандартное отклонение.

Рассмотрим реализацию на python.

Способ 1: Ручная реализация

```
import numpy as np

# Исходные данные: бюджеты фильмов (млн $)
budgets = np.array([237, 200, 185])

# Вычисляем среднее и стандартное отклонение
mean_val = np.mean(budgets)
std_val = np.std(budgets, ddof=0)  # ddof=0 для популяции, ddof=1 для выборки

# Применяем формулу Z-score
budgets_standardized = (budgets - mean_val) / std_val

print("Исходные бюджеты:", budgets)
print("Среднее:", mean_val)
print("Стд. отклонение:", std_val)
print("Стандартизованные:", budgets_standardized)
# Вывод: [ 0.96  0.04 -1.  ]
```

Способ 2: Использование sklearn.preprocessing.StandardScaler

```
import numpy as np
from sklearn.preprocessing import StandardScaler

# Те же данные
data = np.array([
    [162, 2009, 237],
    [194, 1997, 200],
    [152, 2008, 185]
])

# Создаем scaler
scaler = StandardScaler()

# Обучаем и преобразуем
data_standardized = scaler.fit_transform(data)

print("Стандартизованные данные:")
print(data_standardized)
# Вывод:
# [[ 0.16  1.22  1.19]
#  [ 1.22 -1.22 -0.15]
#  [-1.38  0.    -1.04]]
```

```
# Проверяем: среднее должно быть ~0, стд. отклонение ~1
print("\nСреднее по каждому признаку:", np.mean(data_standardized, axis=0))
print("Стд. отклонение:", np.std(data_standardized, axis=0))
# Вывод: Среднее: [0. 0. 0.]
#        Стд. отклонение: [1. 1. 1.]
```

Аналогично max-min методу, классическим способом реализации z-нормализации является использование функций из модуля sklearn.preprocessing.

Задание

1. Реализуйте функции python (def) нормализации max-min и z-нормализации с использованием sklearn. На вход функция должна принимать данные, возвращать нормализованные.
2. Сравните для фильмов значения расстояний в зависимости от выбора способа нормализации. Сделайте выводы.

Обобщенный метрический классификатор

Рассмотрим все объекты выборки x_i и упорядочим по возрастанию расстояния относительно x совместно с целевыми значениями:

$$\rho(x, x^{(1)}) \leq \rho(x, x^{(2)}) \leq \ldots \leq \rho(x, x^{(l)});$$
$$y^{(1)}, y^{(2)}, \ldots, y^{(l)}$$

(верхний индекс – новый порядок наблюдений в выборке). Тогда математически классификатор в самом общем виде можно записать, следующим образом:

$$a(x, X^l) = \underset{y \in Y}{\operatorname{argmax}} \sum_{i=1}^{l} [y^{(i)} = y] \cdot \omega(i, x),$$

Где $\omega(i, x)$ – вес (степень важности) i-го объекта по отношению к x, связанный с метрикой $\rho(x, x_i)$, он неотрицательный и не возрастает при увеличении i.

То есть объект из обучающей выборки является тем более значимым для определения класса нового исследуемого объекта, чем он ближе к нему. То есть обобщенный метрический классификатор – это алгоритм, который классифицирует объект на основе ближайших к нему объектов из обучающей выборки, используя произвольную метрику расстояния.

Примечание: запись $[y^{(i)} = y]$ – это так называемая нотация Айверсона, которая предполагает запись, где логическое утверждение P берется в квадратные скобки $[P]$. Ее значение равно либо 1, если P выполняется, либо 0 в обратном случае. Например:

- $[2 + 2 = 5] = 0$
- ["Яблоко" == "Яблоко"] $= 1$

Наиболее распространенными методами метрической классификации являются метод ближайшего соседа, метод k ближайших соседей и метод парзеновских окон. Рассмотрим их далее более подробно.

Метод ближайшего соседа (NN)

Метод ближайшего соседа конкретизируем формулу метрического классификатора для простейшего случая, когда мы полагаем:

$$\omega(i, x) = [i \leq 1]$$

То есть, у нас i-й вес принимает единичное значение для самой ближайшей точки, и нулевые – для всех остальных:

$$\omega(i, x) = \begin{cases} 1, & \min\limits_{i} \rho(x, x_i) \\ 0, & \text{иначе} \end{cases}$$

Эта запись означает, что для объекта будет выбран тот же класс, к которому относится ближайший к нему объект. Математически этот алгоритм классификации можно записать так:

$$a(x, X^l) = y^{(1)}$$

Напомню, что a – это обозначение алгоритма машинного обучения. То есть в качестве алгоритма в данном случае является поиск метки класса самого ближайшего объекта к текущему объекту. Рассмотри пример. Вернемся к примеру с фильмами. Предположим, есть данные:

Фильм	**Длительность (мин)**	**Год выпуска**	**Бюджет (млн $)**	**Нравится?**
Avatar	162	2009	237	1
Titanic	194	1997	200	0
The Dark Knight	152	2008	185	1
Inception	148	2010	160	1
Interstellar	169	2014	165	?

В качестве метки класса выступает отношение к фильму (1 – нравится, 0 – не нравится). Задача определить, понравится ли фильм Interstellar текущему пользователю. В данном случае рассматривается упрощенная рекомендательная система.

Для начала нормализуем данные. Для этого применяем Min-Max нормализацию.

Признак 1: Длительность (минуты)

Находим минимум и максимум:

$$X_{min} = 148\text{(Inception)}$$
$$X_{max} = 194\text{(Titanic)}$$
$$\text{Диапазон: } 194 - 148 = 46$$

Нормализуем каждый фильм:

$$\text{Avatar: } \frac{162-148}{46} = \frac{14}{46} \approx 0.304$$
$$\text{Titanic: } \frac{194-148}{46} = \frac{46}{46} = 1.000$$
$$\text{Dark Knight: } \frac{152-148}{46} = \frac{4}{46} \approx 0.087$$
$$\text{Inception: } \frac{148-148}{46} = \frac{0}{46} = 0.000$$
$$\text{Interstellar: } \frac{169-148}{46} = \frac{21}{46} \approx 0.457$$

Признак 2: Год выпуска

Находим минимум и максимум:

$$X_{min} = 1997\text{(Titanic)}$$
$$X_{max} = 2014\text{(Interstellar)}$$
$$\text{Диапазон: } 2014 - 1997 = 17$$

Нормализуем каждый фильм:

$$\text{Avatar: } \frac{2009-1997}{17} = \frac{12}{17} \approx 0.706$$
$$\text{Titanic: } \frac{1997-1997}{17} = \frac{0}{17} = 0.000$$
$$\text{Dark Knight: } \frac{2008-1997}{17} = \frac{11}{17} \approx 0.647$$
$$\text{Inception: } \frac{2010-1997}{17} = \frac{13}{17} \approx 0.765$$
$$\text{Interstellar: } \frac{2014-1997}{17} = \frac{17}{17} = 1.000$$

Признак 3: Бюджет (млн $)

Находим минимум и максимум:

$$X_{min} = 160(\text{Inception})$$
$$X_{max} = 237(\text{Avatar})$$
$$\text{Диапазон: } 237 - 160 = 77$$

Нормализуем каждый фильм:

$$\text{Avatar: } \frac{237-160}{77} = \frac{77}{77} = 1.000$$
$$\text{Titanic: } \frac{200-160}{77} = \frac{40}{77} \approx 0.519$$
$$\text{Dark Knight: } \frac{185-160}{77} = \frac{25}{77} \approx 0.325$$
$$\text{Inception: } \frac{160-160}{77} = \frac{0}{77} = 0.000$$
$$\text{Interstellar: } \frac{165-160}{77} = \frac{5}{77} \approx 0.065$$

Округлим полученные значения до одного знака после запятой для удобства. Получим следующие значения:

Фильм	**Длительность (мин)**	**Год выпуска**	**Бюджет (млн $)**	**Нравится?**
Avatar	0.3	0.7	1.0	**1**
Titanic	1.0	0.0	0.5	**0**
The Dark Knight	0.1	0.6	0.3	**1**
Inception	0.0	0.8	0.0	**1**
Interstellar (Тест)	**0.5**	**1.0**	**0.1**	**?**

В рамках данного примера для простоты рассмотрим расчет Манхэттенского расстояния. Считаем расстояние от **Interstellar** $[0.5, 1.0, 0.1]$до каждого фильма:

Avatar: $d_1 = |\ 0.5 - 0.3\ | + |\ 1.0 - 0.7\ | + |\ 0.1 - 1.0\ | = 0.2 + 0.3 + 0.9 = \mathbf{1.4}$

Titanic: $d_2 = |\ 0.5 - 1.0\ | + |\ 1.0 - 0.0\ | + |\ 0.1 - 0.5\ | = 0.5 + 1.0 + 0.4 = 1.9$

Dark Knight: $d_3 = |\ 0.5 - 0.1\ | + |\ 1.0 - 0.6\ | + |\ 0.1 - 0.3\ | = 0.4 + 0.4 + 0.2 = 1.0$

Inception: $d_4 = |\ 0.5 - 0.0\ | + |\ 1.0 - 0.8\ | + |\ 0.1 - 0.0\ | = 0.5 + 0.2 + 0.1 = 0.8$

На следующем этапе сортируем фильмы по возрастанию значениия расстояния до тестового объекта (фильма **Interstellar).** Получим следующий результат:

1. Inception $(d = 0.8) \rightarrow y_{(1)} = 1$
2. The Dark Knight $(d = 1.0) \rightarrow y_{(2)} = 1$
3. Avatar $(d = 1.4) \rightarrow y_{(3)} = 1$
4. Titanic, $(d = 1.9) \rightarrow y_{(3)} = 0$.

Наиболее близким объектом является фильм Inception, который относится к классу 1. В связи с этим тестовый объект Interstellar может быть отнесен к классу 1.

Однако у такого подхода есть критичный недостаток. Если в обучающей выборке по ошибке появится искаженный пример (выброс) какого-либо класса, то все новые объекты, попавшие в его окрестность, будут ошибочно отнесены к этому же неверному классу.

Этот недостаток исправляется методом k ближайших соседей.

Метод k ближайших соседей (kNN)

Предположим, что необходимо учитывать не одного, а k ближайших соседей. В этом случае вектор весов можно определить по правилу:

$$\omega(i, x) = [i \leq k]$$

То есть, теперь веса $\omega(i, x)$ будут принимать единичные значения для первых k ближайших векторов, а остальные оставаться нулевыми. В результате мы приходим к методу **k ближайших соседей** со следующим алгоритмом классификации:

$$a(x, X^l) = \underset{y \in Y}{\operatorname{argmax}} \sum_{i=1}^{k} \left[y^{(i)} = y\right] (1)$$

Вернемся к предыдущему примеру. Фильмы уже были отсортированы по возрастанию расстояния до искомого объекта. Теперь подставим в формулу алгоритма классификации k ближайших соседей, предполагая, что будем определять класс по 3 соседям, то есть k=3. Тогда отбираем три самые близкие объекты, а именно Inception, The Dark Knight, Avatar (имеют наименьшие расстояния до исследуемого тестового объекта Interstellar).

Рассчитаем количество соседей из трех ближайших, которые относятся к классу 1:

$$\text{Класс 1: } \sum_{i=1}^{k}\left[y^{(i)} = 1\right] = [\text{y}(1) = 1] + [\text{y}(2) = 1] + [\text{y}(3) = 1] = [1 = 1] + [1 = 1] + [1 = 1] = 1 + 1 + 1 = 3$$

Аналогично для класса 0:

$$\text{Класс 0: } \sum_{i=1}^{k}\left[y^{(i)} = 0\right] = [\text{y}(1) = 0] + [\text{y}(2) = 0] + [\text{y}(3) = 0] = [1 = 0] + [1 = 0] + [1 = 0] = 0 + 0 + 0 = 0$$

Далее найдем $\underset{y \in \{1,0\}}{\operatorname{argmax}}$, то есть найдем класс, набравший максимальную сумму:

$$\text{y} = \underset{y \in \{1,0\}}{\operatorname{argmax}} \sum_{i=1}^{k} \{3, 0\} = 1$$

Таким образом согласно алгоритму k ближайших соседей, при k=3, фильм Interstellar понравится пользователю (будет отнесен к классу 1).

Сложность данного метода заключается в подборе оптимального количества соседей.

Однако, данный метод имеет недостатки:

- Неопределенность класса при равном количестве соседей разных классах.
- Отсутствие учета расстояния объекта при определении класса

Данные недостатки могут быть устранены методом парзеновских окон.

Задание

1. Провести аналогичные расчеты методом k ближайших соседей для метрики Евклидового расстояния.

Реализация метода k ближайших соседей на python

Для реализации построения метрического алгоритма классификации методом k ближайших соседей на python используем наиболее широко применяемую библиотеку методов машинного обучения sklearn. Как было указано ранее, scikit-learn (часто сокращается до sklearn) – это самая популярная и стандартная библиотека машинного обучения для языка Python с открытым исходным кодом. Она построена на базе NumPy, SciPy и Matplotlib и предоставляет готовые, оптимизированные инструменты для решения задач Data Science.

Библиотека имеет модульную структуру, где каждый модуль отвечает за свой этап работы с данными или моделью. Основные из них:

- Модели (Алгоритмы):
  - linear_model (линейная/логистическая регрессия);
  - tree, ensemble (деревья решений, случайный лес, градиентный бустинг);
  - svm (метод опорных векторов);
  - naive_bayes, neighbors (наивный Байес, KNN).
- Предобработка данных: preprocessing (масштабирование, кодирование), feature_extraction (работа с текстом/изображениями), impute (заполнение пропусков).
- Работа с данными и оценка:
  - model_selection (разделение данных, кросс-валидация, подбор гиперпараметров);
  - metrics (метрики качества: accuracy, F1, MSE, ROC-AUC и т.д.).
- Обучение без учителя: cluster (кластеризация, например, K-Means), decomposition (снижение размерности, PCA).
- Утилиты: datasets (встроенные наборы данных), pipeline (создание цепочек обработки).

В рамках изучаемой темы потреьуется два модуля библиотеки scikit-learn – это neighbors и preprocessing.

Создадим скрипт построения модели классификации для ирисов Фишера, которые были приведены в начале данной работы на рисунке 1. Для начала импортируем необходимые модули:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

Данные по ирисам будут загружены с помощью утилиты load_iris.

train_test_split – это функция из модуля sklearn.model_selection, которая делит ваш набор данных на две части:

1. **train (обучающая выборка) –** это те данные, на которых модель учится (ищет закономерности).
2. **test (тестовая выборка) –** это те данные, на которых модель *не видит* во время обучения. Они нужны, чтобы в конце честно проверить, как модель справляется с новыми, невиданными данными (оценить способность к обобщению).

StandardScaler – это функция нормализации данных.

Для начала загрузим данные:

```
iris = load_iris()
X, y = iris.data, iris.target
```

Далее произведем нормализацию данных.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Здесь создаётся экземпляр класса StandardScaler, затем исходный набор данных преобразуется в нормализованный.

Затем разделим набор на обучающую и тестовую выборки:

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42
)
```

Для построения моделей в sklearn используется один и тот же подход с точки зрения синтаксиса python. Для начала создается экземпляр класса, затем вызывается функция fit, затем predict. Рассмотрим построение модели kNN.

```
# Создаем классификатор
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

# Обучаем
knn.fit(X_train, y_train)

# Предсказываем
y_pred_knn = knn.predict(X_test)
```

Для того, чтобы оценить качество моделей, необходимо ввести некий показатель, который оценит адекватность предсказаний на имеющихся тестовых данных. Такие показатели называются метрики качества моделей. Однако, обратите внимание, что хоть они называются метриками, по факту они метриками с математической точки зрения не являются (не соответствуют свойствам метрик, описанным выше).

В рамках данной работы рассмотрим простейшую метрику точности классификации accuracy. Как было сказано выше, в рамках sklearn есть модуль metrics. Импортируем указанную метрику:

```
from sklearn.metrics import accuracy_score
```

и рассчитаем для построенной модели:

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print(f"\nТочность k-NN (k=5): {accuracy_knn:}")
```

Задание

1. Построить и сравнить несколько моделей методом ближайших соседей, изменяя количество соседей. Построить график точности при разном количестве соседей.

Метод парзеновских окон

Было бы логично учитывать соседние объекты с учетом расстояния до них: чем оно больше, тем меньший вес должно иметь данное наблюдение (близкие соседи несут больший вклад, чем далекие). Для реализации этой идеи веса $\omega(i,x)$ предлагается выбирать с использованием некоторой невозрастающей функции $K(r)$ (ее еще называют **ядром**):

$$\omega(i,x) = K\left(\frac{\rho(x,x_i)}{h}\right)$$

Причем, часто ее выбирают ограниченной на интервале [0; 1] и симметричной относительно нуля (четной). Для окон параметр h в формуле весов определяет область охвата объектов относительно вектора. Например, если h = 1, то максимальное ненулевое расстояние будет равно единице. Если h взять равным 4, то расстояние (охват) также увеличится в 4 раза.

В результате, мы получаем следующий алгоритм классификации с помощью парзеновского окна:

$$a(x,X^l,h,K) = arg\max_{y\in Y}\sum_{i=1}^{l}[y_i = y]\cdot K\left(\frac{\rho(x,x_i)}{h}\right)$$

В качестве функции ядра часто используют следующие функции:

Ядро $K(r)$	Формула
Епанечникова	$E(r)=\frac{3}{4}\left(1-r^2\right)\cdot[\lvert r\rvert\le 1]$
квартическое	$Q(r)=\frac{15}{16}\cdot\left(1-r^2\right)^2\cdot[\lvert r\rvert\le 1]$
треугольное	$T(r)=(1-\lvert r\rvert)\cdot[\lvert r\rvert\le 1]$
гауссовское	$G(r)=\frac{1}{\sqrt{2\pi}}\exp\left(-\frac{r^2}{2}\right)$
прямоугольное	$\Pi(r)=\frac{1}{2}\cdot[\lvert r\rvert\le 1]$

Рассмотрим пример расчета парзеновского окна на примере предыдущих данных про фильмы. Для удобства счета возьмем простую функцию ядра:

$$K(r) = \begin{cases} 1, \text{если } |u| \leq 1, \\ 0, \text{иначе} \end{cases}$$

Для удобства введем обозначения: Avatar (A), Titanic (T), Dark Knight (DK), Inception (I). Для упрощения расчетов оставим только 2 признака: длительность и бюджет.

Рассчитаем манхэттенское расстояние от тестового Interstellar.

$$d(\text{A}) = |\ 0.5 - 0.3\ | + |\ 0.1 - 1.0\ | = 0.2 + 0.9 = 1.1$$

$$d(T) = |\ 0.5 - 1.0\ | + |\ 0.1 - 0.5\ | = 0.5 + 0.4 = 0.9$$

$$d(DK) = |\ 0.5 - 0.1\ | + |\ 0.1 - 0.3\ | = 0.4 + 0.2 = 0.6$$

$$d(I) = |\ 0.5 - 0.0\ | + |\ 0.1 - 0.0\ | = 0.5 + 0.1 = 0.6$$

Далее рассчитываем значение ядра $K(r)$. Для этого зададим значение h=0.7, в результате получим:

$d(\text{A}) = 1.1 > 0.7 \Rightarrow \cdot\ K\left(\frac{\rho(x,x_i)}{h}\right) = 0$ (не учитывается)

$d(T) = 0.9 > 0.7 \Rightarrow \cdot\ K\left(\frac{\rho(x,x_i)}{h}\right) = 0$(не учитывается)

$d(DK) = 0.6 \leq 0.7 \Rightarrow \cdot\ K\left(\frac{\rho(x,x_i)}{h}\right) = 1$(учитывается)

$d(I) = 0.6 \leq 0.7 \Rightarrow \cdot\ K\left(\frac{\rho(x,x_i)}{h}\right) = 1$(учитывается)

Рассчитаем количество соседей из трех ближайших, которые относятся к классу 1:

Класс 1: $\sum_{i=1}^{k}\left[y^{(i)} = 1 \cdot K\left(\frac{\rho(x,x_i)}{h}\right)\right] = [\text{y(A)} = 1] \cdot 0 + [\text{y(T)} = 1] \cdot 0 + [\text{y(DK)} = 1] \cdot 1 + [\text{y(I)} = 1] \cdot 1 = [1 = 1] \cdot 0 + [0 = 1] \cdot 0 + [1 = 1] \cdot 1 + [1 = 1] \cdot 1 = 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 = 0 + 0 + 1 + 1 = 2$

Аналогично для 0:

Класс 0: $\sum_{i=1}^{k}\left[y^{(i)} = 0\right] \cdot K\left(\frac{\rho(x,x_i)}{h}\right) = [\text{y(A)} = 0] \cdot 0 + [\text{y(T)} = 0] \cdot 0 + [\text{y(DK)} = 0] \cdot 1 + [\text{y(I)} = 0] \cdot 1 = [1 = 0] \cdot 0 + [0 = 0] \cdot 0 + [1 = 0] \cdot 1 + [1 = 0] \cdot 1 = 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 = 0 + 0 + 0 + 0 = 0$

Далее найдем $\underset{y\in\{1,0\}}{\text{argmax}}$, то есть найдем класс, набравший максимальную сумму:

$$\text{y} = \underset{y\in\{1,0\}}{\text{argmax}} \sum_{i=1}^{k} \{2, 0\} = 1$$

При $h = 0.7$ выбранное ядро фактически работает как **2-NN** (учитываются только 2 ближайших соседа: DK и I).

Задание

1. Провести аналогичные расчеты методом парзеновских окон для метрики Евклидового расстояния.

Реализация метода k ближайших соседей на python

В sklearn нет прямого классификатора Парзена, но есть оценка плотности. Поэтому Применим следующий подход:

```
classes = np.unique(y_train)
bandwidth = 1.0  # ширина окна (h)
```

classes – это переменная, которая содержит уникальные метки классов (массив [0, 1, 2]).

bandwidth – самый важный гиперпараметр метода Парзена (ширина окна сглаживания h). Он определяет, насколько "широкими" будут гауссовы колокола вокруг каждой точки.

- Если h слишком мало — модель переобучится (плотность будет рваной).
- Если h слишком велико — модель недообучится (все классы сольются в одно пятно).

Далее реализуем алгоритм парзеновского окна:

```
kde_models = {}
for c in classes:
    kde = KernelDensity(bandwidth=bandwidth, kernel='gaussian')
    # Обучаем на объектах только текущего класса
    kde.fit(X_train[y_train == c])
    kde_models[c] = kde
```

- Создаем пустой словарь, где будем хранить по одной модели KDE для каждого класса.
- Запускаем цикл по классам (0, 1, 2).
- Создаем модель KDE с гауссовым ядром и заданной шириной окна.
- X_train[y_train == c]: ключевая строка. Мы берем из обучающей выборки *только те объекты, которые принадлежат текущему классу c*.
- kde.fit(...): "обучаем" модель. На самом деле KDE не учит веса, она просто "запоминает" точки и строит вокруг них гауссовы колоколы, суммируя их. Мы получили оценку плотности $P(x \mid class = c)$.
- Сохраняем обученную модель в словарь.

Далее считаем предсказанное значение (метку класса):

```
y_pred_parzen = []
for x in X_test:
    # Вычисляем логарифм плотности для каждого класса
    log_probs = {}
    for c in classes:
        # score_sample возвращает log-вероятность
        log_prob = kde_models[c].score_samples([x])[0]
        log_probs[c] = log_prob

    # Выбираем класс с максимальной плотностью
    pred_class = max(log_probs, key=log_probs.get)
    y_pred_parzen.append(pred_class)
```

```
y_pred_parzen = np.array(y_pred_parzen)

# Оцениваем качество
accuracy_parzen = accuracy_score(y_test, y_pred_parzen)
```

Задание

1. Провести аналогичные расчеты методом парзеновских окон для различных h. Сравните результаты.

Заключение

В результате данного занятия были изучены следующие методы:

- метрики раастояний в машинном обучении;
- метод ближайшего соседа;
- метод k-ближайших соседей;
- метод парзеновского окна;
- метод потенциальных функций

Задание

1. Загрузить данные diabetes_data с использованием pandas.
2. Провести нормализацию.
3. Построить модели классификации kNN и методом парзеновского окна.
4. Рассчитать метрику качества.

Задание

Самостоятельная работа

1. Выберите набор данных для классификации
2. Построить метрические классификаторы, изученные в рамках данного задания.

Контрольные вопросы

1. Что такое метрика расстояния?
2. Какие свойства метрик расстояния существуют?
3. Евклидово расстояние
4. Манхэттенское расстояние
5. Метрика Минковского
6. Понятие метрического классификатора
7. Классификация методом ближайшего соседа
8. Классификация методом k ближайших соседей
9. Классификация методом парзеновских окон.